

Problem L – Legendary Sort

The Legendary Huron is learning about sorting algorithms. After studying Bubble Sort and Bogo Sort, he became so enthusiastic that he invented his own algorithm: the *Legendary Sort*. This algorithm sorts an array and consists of two phases:

1. First, rotate the array to the left by k positions ($0 \leq k < n$). Formally, the array $[a_1, a_2, \dots, a_n]$ becomes $[a_{k+1}, a_{k+2}, \dots, a_n, a_1, a_2, \dots, a_k]$. For example, the array $[1, 2, 3, 4, 5]$ becomes $[3, 4, 5, 1, 2]$ when $k = 2$.
2. Then, while the array is not sorted, repeatedly perform the following operation: find an index i ($1 \leq i < n$) such that $a_i > a_{i+1}$ and swap these two elements. If there are multiple valid indices i , any of them can be chosen.

The Legendary Huron wants to test his new algorithm in different scenarios. For that, he considers a **permutation** $[p_1, p_2, \dots, p_n]$ of length n , and processes q queries on this permutation. Each query is one of the following types:

1. Given an index i ($1 \leq i < n$), swap the elements p_i and p_{i+1} .
2. Given an integer k ($0 \leq k < n$), execute the Legendary Sort on the current permutation, rotating it by k positions to the left in the first phase. Compute the minimum number of swaps that can be performed in the second phase. Note that the rotation of the first phase **does not** persist for future queries.

Help the Legendary Huron to process these queries.

Input

The first line contains two integers n and q ($2 \leq n \leq 2 \cdot 10^5$ and $1 \leq q \leq 2 \cdot 10^5$) — the length of the permutation and the number of queries.

The second line contains n **distinct** integers $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq n$) — the permutation.

Each of the next q lines describes a query. Each line begins with an integer t ($t \in \{1, 2\}$) — the type of the query. If $t = 1$, an integer i ($1 \leq i < n$) follows. If $t = 2$, an integer k ($0 \leq k < n$) follows.

Output

For each query of type $t = 2$, print an integer — the minimum number of times that the swap operation can be performed in the second phase.

Sample input 1 5 5 1 2 3 4 5 2 0 2 2 1 2 2 0 2 2	Sample output 1 0 6 1 5
Sample input 2 5 2 3 4 5 1 2 2 3 2 0	Sample output 2 0 6